

Projet AWS

Job 1 – Déploiement d'une application web scalable sur AWS (ou LocalStack)

0.1 Installer Docker

0.3 Installer AWS CLI

ÉTAPE 1 – Créer une instance EC2

Objectif

Avoir **une instance EC2 fonctionnelle** avec une **application web simple**.

```
sudo systemctl enable docker
sudo systemctl start docker
```

```
sudo usermod -aG docker $USER
newgrp docker
```

```
sudo apt install -y awscli
```

0.4 Configurer AWS CLI pour LocalStack

```
aws configure
```

Créer un alias pratique :

```
echo "alias awslocal='aws --endpoint-url=http://localhost:4566'" >> ~/.bashrc
source ~/.bashrc
```

Comment vérifier CORRECTEMENT que LocalStack fonctionne

```
curl http://localhost:4566/_localstack/health
```

```
{  
  "services": {  
    "ec2": "running",  
    "s3": "running",  
    "elb": "running",  
    "autoscaling": "running"  
  }  
}
```

Installation de la CLI LocalStack

```
sudo apt update  
sudo apt install -y python3-pip
```

```
pip3 install localstack
```

Créer le Security Group

```
awslocal ec2 create-security-group \  
  --group-name web-sg \  
  --description "Web Security Group"
```

Ajouter les règles d'accès

```
awslocal ec2 authorize-security-group-ingress \  
  --group-name web-sg \  
  --protocol tcp \  
  --port 22 \  
  --cidr 0.0.0.0/0
```

```
awslocal ec2 authorize-security-group-ingress \  
  --group-name web-sg \  
  --protocol tcp \  
  --port 80 \  
  --cidr 0.0.0.0/0
```

Étape 2 : Créer une instance EC2

```
cat << EOF > user-data.sh
#!/bin/bash
apt update -y
apt install -y apache2
echo "<h1>Serveur EC2 - Auto Scaling</h1>" > /var/www/html/index.html
systemctl start apache2
systemctl enable apache2
EOF
```

Lancer l'instance

```
awslocal ec2 run-instances \
  --image-id ami-12345678 \
  --instance-type t2.micro \
  --key-name web-key \
  --security-groups web-sg \
  --user-data file://user-data.sh \
  --count 1
```

Vérifier l'instance

```
awslocal ec2 describe-instances
```

Erreur connu pour l'instance :

matteo@localstackAWS:~\$ awslocal ec2 describe-instances Could not connect to the endpoint URL: "http://localhost:4566/"

Solution :

Vérifier tous les conteneurs

CONTAINER ID	IMAGE	STATUS	NAMES
0304f60229c3	localstack/localstack	Exited	localstack

Si STATUS est Exited → le conteneur est **arrêté**

S'il est en autre état → note le statut

Supprimer le conteneur existant :

```
docker rm localstack
```

Si Docker te demande confirmation → ok

Si le conteneur était actif → il faut d'abord le stopper :

```
docker stop localstack  
docker rm localstack
```

Relancer LocalStack :

```
docker run -d \  
  --name localstack \  
  -p 4566:4566 \  
  -p 4571:4571 \  
  localstack/localstack
```

Tester le health endpoint

```
curl http://localhost:4566/_localstack/health
```

Créer une AMI (obligatoire)

```
awslocal ec2 create-image \  
  --instance-id i-0abcd1234 \  
  --name "web-app-ami" \  
  --no-reboot
```

Créer un Launch Template

```
awslocal ec2 create-launch-template \  
  --launch-template-name web-template \  
  --launch-template-data '{  
    "ImageId": "ami-0aaabbbccc",  
    "InstanceType": "t2.micro",  
    "SecurityGroupIds": ["sg-41d8f44eadb6c5049"],  
    "UserData": "'$(base64 -w 0 user-data.sh)'"  
  }'
```

Vérifier

```
awslocal ec2 describe-launch-templates
```

Créer l'Auto Scaling Group

```
awslocal autoscaling create-auto-scaling-group \  
  --auto-scaling-group-name web-asg \  
  --launch-template LaunchTemplateName=web-template,Version=1 \  
  --min-size 1 \  
  --max-size 3 \  
  --desired-capacity 1 \  
  --vpc-zone-identifier subnet-XXXXXXX
```

Vérifier l'ASG

```
awslocal autoscaling describe-auto-scaling-groups
```

Job 2

Objectif: Héberger un site web statique avec une distribution CloudFront pour une meilleure performance et sécurité.

Créer un bucket S3

```
awslocal s3 mb s3://mon-site-statique
```

Créer un fichier HTML simple

```
mkdir site  
nano site/index.html
```

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Mon site S3</title>  
</head>  
<body>  
  <h1>Site statique sur S3</h1>  
  <p>Déployé avec AWS</p>  
</body>  
</html>
```

Envoyer le site dans S3

```
awslocal s3 cp site/ s3://mon-site-statique/ --recursive
```

Activer l'hébergement statique S3

```
awslocal s3 website s3://mon-site-statique/ \
--index-document index.html
```

Rendre le contenu public

```
awslocal s3api put-bucket-policy \
--bucket mon-site-statique \
--policy '{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::mon-site-statique/*"
  }]
}'
```

Tester l'accès S3 (LocalStack)

```
http://localhost:4566/mon-site-statique/index.html
```

CloudFront

```
aws cloudfront create-distribution \
--origin-domain-name mon-site-statique.s3.amazonaws.com
```

Ce que CloudFront apporte

- Mise en cache du contenu
- Réduction de la latence
- HTTPS automatique
- Protection basique contre attaques

Job 3

Objectif: Déployer une base de données relationnelle avec des sauvegardes automatiques pour un stockage sécurisé des données.

Créer un rôle IAM

```
awslocal iam create-role \  
  --role-name rds-backup-role \  
  --assume-role-policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [{  
      "Effect": "Allow",  
      "Principal": {"Service": "rds.amazonaws.com"},  
      "Action": "sts:AssumeRole"  
    }]  
  }'
```

Créer l'instance RDS

```
aws rds create-db-instance \  
  --db-instance-identifier mon-rds \  
  --db-instance-class db.t2.micro \  
  --engine mysql \  
  --master-username admin \  
  --master-user-password motdepasse123 \  
  --allocated-storage 20 \  
  --backup-retention-period 7 \  
  --vpc-security-group-ids sg-41d8f44eadb6c5049
```

PostgreSQL exemple :

```
aws rds create-db-instance \  
  --db-instance-identifier mon-rds \  
  --db-instance-class db.t2.micro \  
  --engine postgres \  
  --master-username admin \  
  --master-user-password motdepasse123 \  
  --allocated-storage 20 \  
  --backup-retention-period 7 \  
  --vpc-security-group-ids sg-41d8f44eadb6c5049
```

Vérifier l'instance

```
aws rds describe-db-instances
```

Configurer snapshots automatiques

```
aws rds create-db-snapshot \  
  --db-instance-identifier mon-rds \  
  --db-snapshot-identifier mon-snapshot-001
```

Job 4

Objectif: Créer une API RESTful sans serveur qui répond à des requêtes HTTP.

Principe de fonctionnement

```
Client (navigateur / curl)
  ↓
API Gateway (URL publique)
  ↓
Lambda (code Python / Node.js)
  ↓
Réponse JSON
```

Créer une fonction Lambda (Python)

```
mkdir lambda-api
cd lambda-api
nano lambda_function.py
```

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": "Bonjour depuis Lambda 🚀"
    }
```

Créer l'archive ZIP

```
zip function.zip lambda_function.py
```

Créer un rôle IAM pour Lambda

```
aws iam create-role \
  --role-name lambda-basic-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "lambda.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'
```


Ajouter la permission CloudWatch Logs :

```
aws iam attach-role-policy \  
  --role-name lambda-basic-role \  
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
```

Créer la fonction Lambda

```
aws lambda create-function \  
  --function-name hello-api \  
  --runtime python3.12 \  
  --role arn:aws:iam::ACCOUNT_ID:role/lambda-basic-role \  
  --handler lambda_function.lambda_handler \  
  --zip-file fileb://function.zip
```

Créer une API Gateway REST

```
aws apigateway create-rest-api --name "hello-api"
```

Récupérer le Root Resource ID

```
aws apigateway get-resources --rest-api-id a1b2c3d4
```

Créer une ressource

```
aws apigateway create-resource \  
  --rest-api-id a1b2c3d4 \  
  --parent-id ROOT_ID \  
  --path-part hello
```

Créer la méthode GET

```
aws apigateway put-method \  
  --rest-api-id a1b2c3d4 \  
  --resource-id RESOURCE_ID \  
  --http-method GET \  
  --authorization-type NONE
```

Lier API Gateway à Lambda

```
aws apigateway put-integration \  
  --rest-api-id a1b2c3d4 \  
  --resource-id RESOURCE_ID \  
  --http-method GET \  
  --type AWS_PROXY \  
  --integration-http-method POST \  
  --uri arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:ACCOU
```

Autoriser API Gateway à appeler Lambda

```
aws lambda add-permission \  
  --function-name hello-api \  
  --statement-id apigateway-test \  
  --action lambda:InvokeFunction \  
  --principal apigateway.amazonaws.com
```

Déployer l'API

```
aws apigateway create-deployment \  
  --rest-api-id a1b2c3d4 \  
  --stage-name prod
```

Tester l'API

```
https://a1b2c3d4.execute-api.us-east-1.amazonaws.com/prod/hello
```

◆ Job 1 — Application web scalable sur EC2

🎯 Objectif

Déployer une application web sur EC2 avec mise à l'échelle automatique et répartition de charge.

🛠 Services utilisés

- EC2
- VPC / Subnet
- Security Groups
- Auto Scaling Group
- Application Load Balancer

✅ Réalisé

- Création d'un VPC et d'un Subnet
- Création d'un Security Group (ports 22 et 80)
- Déploiement d'une instance EC2 avec installation automatique de l'application web (User Data)
- Création d'une AMI
- Création d'un Launch Template

⚠ Limitation

- Auto Scaling Group et Load Balancer non disponibles dans LocalStack Community

✍ À écrire dans le rapport

L'application web a été déployée sur une instance EC2 sécurisée par un Security Group. Une AMI et un Launch Template ont été créés afin de permettre une mise à l'échelle automatique. L'Auto Scaling Group et le Load Balancer n'ont pas pu être déployés localement en raison des limitations de LocalStack Community, mais l'architecture est prête pour un déploiement sur AWS réel.

💰 Coût

- EC2 t2.micro : 750 heures gratuites / mois

◆ Job 2 — Hébergement d'un site statique avec CloudFront

🎯 Objectif

Héberger un site statique rapide et sécurisé.

🛠 Services utilisés

- S3
- CloudFront
- IAM

✅ Réalisé

- Création d'un bucket S3
- Déploiement d'un site HTML statique
- Activation de l'hébergement web S3
- Configuration de l'accès public

⚠️ Limitation

- CloudFront non supporté en LocalStack Community

✍️ À écrire dans le rapport

Un site web statique a été hébergé dans un bucket S3 avec l'hébergement web activé. Une distribution CloudFront a été prévue afin d'améliorer les performances et la sécurité, mais n'a pas pu être déployée localement en raison des limitations de LocalStack Community.

💰 Coût

- S3 + CloudFront : inclus dans le Free Tier

◆ Job 3 — Base de données relationnelle sécurisée

🎯 Objectif

Déployer une base de données relationnelle avec sauvegardes automatiques.

🧰 Services utilisés

- Amazon RDS (MySQL / PostgreSQL)
- IAM

⚠️ Limitation

- RDS non supporté en LocalStack Community

✅ Réalisé

- Préparation des commandes RDS
- Définition des paramètres :
 - sauvegardes automatiques
 - snapshots
 - sécurité réseau (VPC + Security Group)

✍️ À écrire dans le rapport

Une instance RDS MySQL/PostgreSQL serait déployée dans le VPC avec des sauvegardes automatiques et des snapshots réguliers. En raison des limitations de LocalStack Community, la création effective de l'instance RDS n'a pas été possible localement.

Coût

- RDS db.t2.micro : 750 heures gratuites / mois

◆ Job 4 — API RESTful Serverless

Objectif

Créer une API REST sans serveur, scalable et économique.

Services utilisés

- AWS Lambda
- API Gateway
- CloudWatch Logs

Réalisé

- Développement d'une fonction Lambda (Python)
- Création d'une API REST via API Gateway
- Liaison API Gateway → Lambda
- Activation automatique des logs CloudWatch

À écrire dans le rapport

Une API RESTful serverless a été mise en place à l'aide d'AWS Lambda et API Gateway. Chaque requête HTTP déclenche une fonction Lambda, et les journaux d'exécution sont stockés automatiquement dans CloudWatch.

Coût

- 1 million de requêtes gratuites / mois (Lambda + API Gateway)

Conclusion générale

Ce projet a permis de concevoir une architecture cloud complète, combinant services IaaS, PaaS et Serverless. Malgré certaines limitations liées à l'utilisation de LocalStack Community, l'ensemble de l'infrastructure est prêt pour un déploiement réel sur AWS, tout en respectant les bonnes pratiques de sécurité, de scalabilité et de maîtrise des coûts.